

Funcționalități suplimentare adăugate:

- Buton pentru stingerea luminilor din scenă principală;
- Atribuirea funcționalității butonului ESC de la tastatură;
- Timer;
- Cod alternativ pentru mișcarea sferei, după cameră.

1. Buton pentru stingerea luminilor din scena principală:

Secvența de cod:

```
using UnityEngine;
using UnityEngine.UI;

public class LightToggleController : MonoBehaviour
{
    public GameObject[] lights; // Array of light GameObjects to toggle
    private bool lightsOn = true; // State of the lights

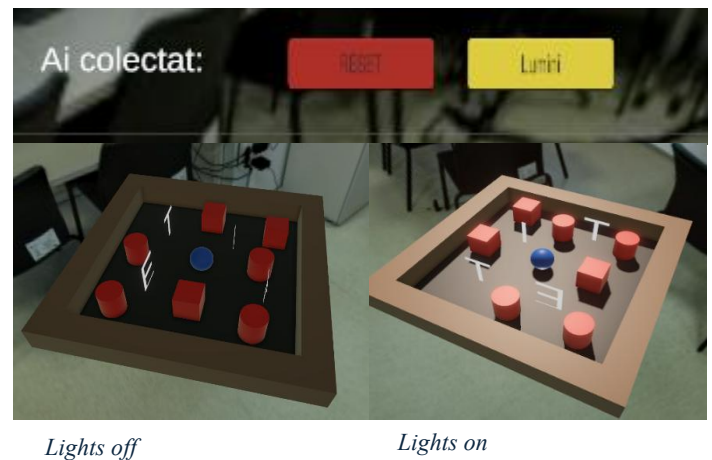
    public Button toggleLightsButton; // Reference to the toggle button

    void Start()
    {
        if (toggleLightsButton != null)
        {
            // Assign the ToggleLights function to the button's onClick event
            toggleLightsButton.onClick.AddListener(ToggleLights);
        }
    }

    void ToggleLights()
    {
        lightsOn = !lightsOn; // Toggle the state

        // Loop through all lights and set their active state
        foreach (GameObject light in lights)
        {
            light.SetActive(lightsOn);
        }

        Debug.Log(lightsOn ? "Lights turned ON" : "Lights turned OFF");
    }
}
```



Scurtă descriere: La apăsarea butonului “Lumini” se vor activa/dezactiva toate luminile din scena.

2. Atribuirea funcționalității butonului ESC de la tastatură:

Secvența de cod:

```
private void ExitGame()
{
    Debug.Log("Exiting game...");
}

#if UNITY_EDITOR
    UnityEditor.EditorApplication.isPlaying = false; // Stop play mode in the Unity Editor
#else
    Application.Quit(); // Quit the application in a standalone build
#endif
}

void Update()
{
    // Check if the Esc key is pressed
    if (Input.GetKeyDown(KeyCode.Escape))
    {
        ExitGame(); // Call the ExitGame method when Esc is pressed
    }
}
```

Scurtă descriere: La apăsarea butonului ESC de la tastatură jocul se va oprii.

3. Timer:

Secvența de cod:

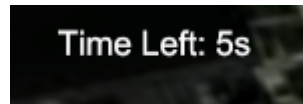
```
using UnityEngine;
using UnityEngine.UI;
using UnityEngine.SceneManagement;

public class TimerController : MonoBehaviour
{
    public Text timerText; // Reference to the Legacy Text component
    public float timeRemaining = 15f; // Time in seconds
    private bool timerRunning = true; // Control if the timer is active

    void Update()
    {
        if (timerRunning)
        {
            if (timeRemaining > 0)
            {
                timeRemaining -= Time.deltaTime;
                UpdateTimerDisplay();
            }
            else
            {
                timerRunning = false;
                timeRemaining = 0;
                UpdateTimerDisplay();
                ResetScene(); // Reset the current scene when the timer ends
            }
        }
    }

    void UpdateTimerDisplay()
    {
        // Update the text display to show remaining time rounded to the nearest integer
        if (timerText != null)
        {
            timerText.text = "Time Left: " + Mathf.Ceil(timeRemaining).ToString() + "s";
        }
    }

    void ResetScene()
    {
        Debug.Log("Time is up! Resetting scene...");
        SceneManager.LoadScene(SceneManager.GetActiveScene().buildIndex); // Reload the current scene
    }
}
```



Scurtă descriere: Am adăugat UI – Legacy text la care am atașat scriptul cu funcționalitățile acestuia. Jucatorul are un timp limită de 20 secunde, după trecerea timpului scena se va reseta.

4. Cod alternativ pentru mișcarea sferei, după camera:

Secvența de cod:

```
public Camera mainCamera; // Reference to the main camera for relative movement

// Ensure the main camera is assigned
if (mainCamera == null)
{
    mainCamera = Camera.main;
}

private void FixedUpdate()
{
    float movementHorizontal = Input.GetAxis("Horizontal");
    float movementVertical = Input.GetAxis("Vertical");

    // Get camera's forward and right directions
    Vector3 cameraForward = mainCamera.transform.forward;
    Vector3 cameraRight = mainCamera.transform.right;

    // Remove the Y component to keep movement on the horizontal plane
    cameraForward.y = 0;
    cameraRight.y = 0;

    // Normalize the vectors to ensure consistent speed
    cameraForward.Normalize();
    cameraRight.Normalize();

    // Calculate movement relative to the camera
    Vector3 movement = (cameraForward * movementVertical + cameraRight * movementHorizontal) * speed * Time.deltaTime;

    // Apply the movement to the Rigidbody
    playerRigidbody.AddForce(movement);

    // If you press Space bar in Runtime your sphere GameObject jumps
    if (Input.GetButton("Jump"))
    {
        playerRigidbody.AddForce(transform.up * m_Thrust);
    }
}
```

Scurtă descriere: Am adăugat Codul ajustează mișcarea jucătorului pentru a fi relativă la orientarea camerei, folosind direcțiile camerei pentru a calcula deplasarea pe plan orizontal, asigurând astfel ca mișcarea să urmeze perspectiva camerei.